

Kamino451

by Florian Buetow

Kamino. I'm not familiar with it. Is it in the Republic?

No, no. It's beyond the Outer Rim. I'd say about, 12 parsecs outside the Rishi Maze.

Obi-Wan Kenobi and Dexter Jettster in
Star Wars: Episode II - Attack of the Clones



Kamino

TODO: Kamino is an aquatic planet ...

Fahrenheit 451

TODO: A Sci-Fi Novel by Ray Bradbury about a dystopian future about censorship where the books that humans are allowed to read are restricted.

Kamino451

TODO: Combining the idea of cloning agents and tailoring the context they are allowed to receive.

Speaker



Florian Buetow

Software Consultant at Xebia

LinkedIn: [linkedin.com/in/fbuetow](https://www.linkedin.com/in/fbuetow)

GitHub: github.com/florianbuetow

Blog: cracking-ai-engineering.com





Claude Code v2.1.169

Sonnet 4.6 with high effort · Claude Max
/Users/flo

What Is Kamino451?



LIVE DEMO

Agent Factory Features

- Filesystem based (`.kamino/`)
- Blueprints for "cloning" tested agents
- Blueprints for creating ad-hoc agents
- Task complexity ranker
- Automatic task decomposition
- Automatic model to agent mapping
- A `factory` plugin for Claude/codex
- Task agnostic agent orchestrator
- Agent evaluation framework

Typical Workflow 1

- State your task
- Get back agents
- Launch the agents

Behind the scenes

- Task decomposition
- Task to agent mapping
- Task to model mapping

Example

Using the factory skill



```
/factory Write me an article based on demo/bratwurst-wars.md as the source material, following demo/styleguide.md.
```



/factory Write me an article based on demo/bratwurst-wars.md as the source material, following demo/styleguide.md. But I also want you to fact check and review the article for me and then present me the final results as files.



```
/factory lets use the reviews to iterate on our current article draft, then perform another check.
```

/delegate to opus the following command verbatim without any additions or redactions: "/factory to orchestrate the following task: /factory write me an article based on demo/bratwurst-wars.md as the source material, following demo/styleguide.md. But I also want you to fact check and review the article for me and then present me the final results as files."

Typical Workflow 2

- Evaluate your agents (improve task to agent mapping)
- Improve your agents (improve model to agent mapping)

Behind the scenes

- Collect data
- Run auto-research
- Version improvements
- Generate report

PRESENTED BY DYNAMOUS & KIRO

Kiro Hackathon

\$17,000

in prizes

The results are in and the winners have been announced. **Scroll down to see all 16 winners!**

Presented by Dynamous & AWS Kiro

 Judging: Complete  Winners: Announced!

Watch Winners Announcement ▶

Join Community 

The hackathon is complete — thank you to all participants and congratulations to the winners!

Specification-Driven Development



Let's build

Plan, search, or build anything

Vibe

Chat first, then build. Explore ideas and iterate as you discover needs.

Spec

Plan first, then build. Create requirements and design before coding starts.

Great for:

- Thinking through features in-depth
- Projects needing upfront planning
- Building features in a structured way

First Attempt: Specification-Driven

- R1: The focused pane's title must be rendered with ItemSelected styling (blue bg, white fg).
- R2: The inactive pane's title must retain default styling (bold white text, no background).
- R3: Highlighting must update immediately when Tab is pressed, with no perceptible delay.
- R4: Highlighting must not break the top border alignment – the junction character must remain aligned with the separator.

Survived planning. Did not survive implementation.

Attempt 2: Tutorials as Specification

- **Tutorial 1** - CLI for individual files
- **Tutorial 2** - groups of files (collections)
- **Tutorial 3** - TUI, with mockups

Much more detailed than a SPECS.md.

Tutorial 1 - Protect a Single File

```
$ guard init 0644 root wheel
$ touch test.txt
$ guard add file test.txt

$ cat .guardfile
config:
  guard_mode: "0644"
files:
  - path: ./test.txt
    mode: "0644"
    guard: false

$ sudo guard toggle file test.txt
Guard enabled for test.txt
```

User stories with inputs and outputs.

Tutorial 2 - Protect a Collection

```
$ guard create alice
$ guard update alice add alice1.txt alice2.txt shared.txt

$ guard show collection alice
[-] collection: alice (3 files)

$ sudo guard enable collection alice
Guard enabled for collection alice
Guard enabled for alice1.txt
Guard enabled for alice2.txt
Guard enabled for shared.txt
```

Last-operation-wins on shared files; conflict detection on ambiguous toggles.

Tutorial 3 - The TUI Mockup

Files	Collections
<pre>> docs [G] alice1.txt [G] alice2.txt [-] bob1.txt [G] shared.txt</pre>	<pre>[G] alice [-] bob</pre>
<pre>↑↓:Navigate ↔:Expand Space:Toggle Tab:Switch /:Search Q:Quit</pre>	

Screen capture as specification.

The AI Said It Was Done

```
PLACEHOLDER (resolved by C1):  
  "The implementation is now complete."  
  $ just test  
  FAIL: TestGuardAdd_PreservesPermissions
```

But the tests were still failing.

"Yes, you are absolutely right!"

Tests as Machine-Readable Spec

```
test_add_positive() {  
  # Setup  
  $GUARD_BIN init 000 $(whoami) $(id -gn)  
  touch test1.txt  
  initial_perms=$(get_file_permissions test1.txt)  
  
  # Run  
  $GUARD_BIN add test1.txt  
  
  # Assert  
  assert_exit_code $? 0 "guard add should succeed"  
  file_in_registry "$(pwd)/test1.txt" || fail "not in registry"  
  assert_equals "false" "$(get_guard_flag $(pwd)/test1.txt)"  
  assert_equals "$initial_perms" "$(get_file_permissions test1.txt)"  
}
```

Same specs, different "encoding"

The Bootstrap Script

```
chmod -w tests/test-tui-search-001.sh  
chflags uchg internal/security/path_validator.go
```

Protecting the first tests and code files manually

Corollary: What Worked, What Didn't

Models

- Some followed specs better than others
- Top tier $\neq$ guaranteed
- Over-engineering varies by family
- Deviate from the best specifications

Harnesses

- Claude Code CLI $\rightarrow$ tightest loop
- Kiro IDE $\rightarrow$ heavier spec ceremony
- Harness mattered more than model

AI Code, Characteristic Mistakes

- Defaults that silently mask missing config
- Fallbacks that hide the real failure
- `log.Print(err)` and keep going
- Over-engineering a 30-line script into a framework
- Moving files between packages on every refactor

More Static Guardrails

- **gofmt** - formatting
- **golangci-lint** - meta-linter suite
- **go vet / staticcheck** - bug patterns
- **gocyclo** - cyclomatic complexity
- **gocognit** - cognitive complexity
- **semgrep** - forbidden patterns
- **gosec** - security
- **layer tests** - architectural contracts
- **shell + tmux tests** - behavior
- **just ci** - runs them all

Semgrep - Restrict Access Patterns

```
- id: registry-load-restricted
  message: |
    Direct calls to registry.Load() are restricted to the
    security layer. All loads must go through Security,
    which performs path validation and tamper detection.
  severity: ERROR
  pattern: $REG.Load()
  paths:
    exclude:
      - "**/*_test.go"
      - "**/internal/security/security.go"
```

One rule. Architecture enforced statically, instead of by hope.

golangci-lint - Many Checks, One Wall

```
linters:  
  enable:  
    - errcheck      # unchecked errors  
    - govet          # suspicious constructs  
    - staticcheck    # bug patterns  
    - ineffassign    # ineffective assignments  
    - unused         # dead code  
    - gosec          # security issues  
    - revive         # style  
  
issues:  
  max-issues-per-linter: 0  
  max-same-issues: 0
```

Linters are guardrails the AI can't argue with.

Complexity Gates

```
# Cyclomatic – branching depth  
gocyclo -over 10 .
```

```
# Cognitive – how hard a human (or AI) has to read it  
gocognit -over 15 .
```

Two thresholds. Build red until the AI splits the function.

Architecture by Contract

Enforcing boundaries and dependencies.

Architectural Unit Tests

```
func TestLayering_NoTuiFilesystemImports(t *testing.T) {  
    tuiDir := filepath.Join(repoRoot(t), "internal", "tui")  
    forbidden := []string{"/internal/filesystem"}  
    assertNoForbiddenStrings(t, tuiDir, forbidden)  
}  
  
func TestLayering_NoPrintingInManagerOrFilesystem(t *testing.T) {  
    forbidden := []string{"fmt.Print"}  
    assertNoForbiddenStrings(t,  
        filepath.Join(repoRoot(t), "internal", "manager"), forbidden)  
    assertNoForbiddenStrings(t,  
        filepath.Join(repoRoot(t), "internal", "filesystem"), forbidden)  
}
```

The AI moves files between layers on every refactor. These tests stop it.

Enforcing Quality Gates

Stop Hook

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "just ci-quiet"
          }
        ]
      }
    ]
  }
}
```

Coding agent gets feedback every turn until `just ci-quiet` passes.

Git Pre-Commit Hook

```
# .githooks/pre-commit  
#!/bin/bash  
just ci-quiet
```

Enable once with `git config core.hooksPath .githooks`. Commit is blocked if CI fails.

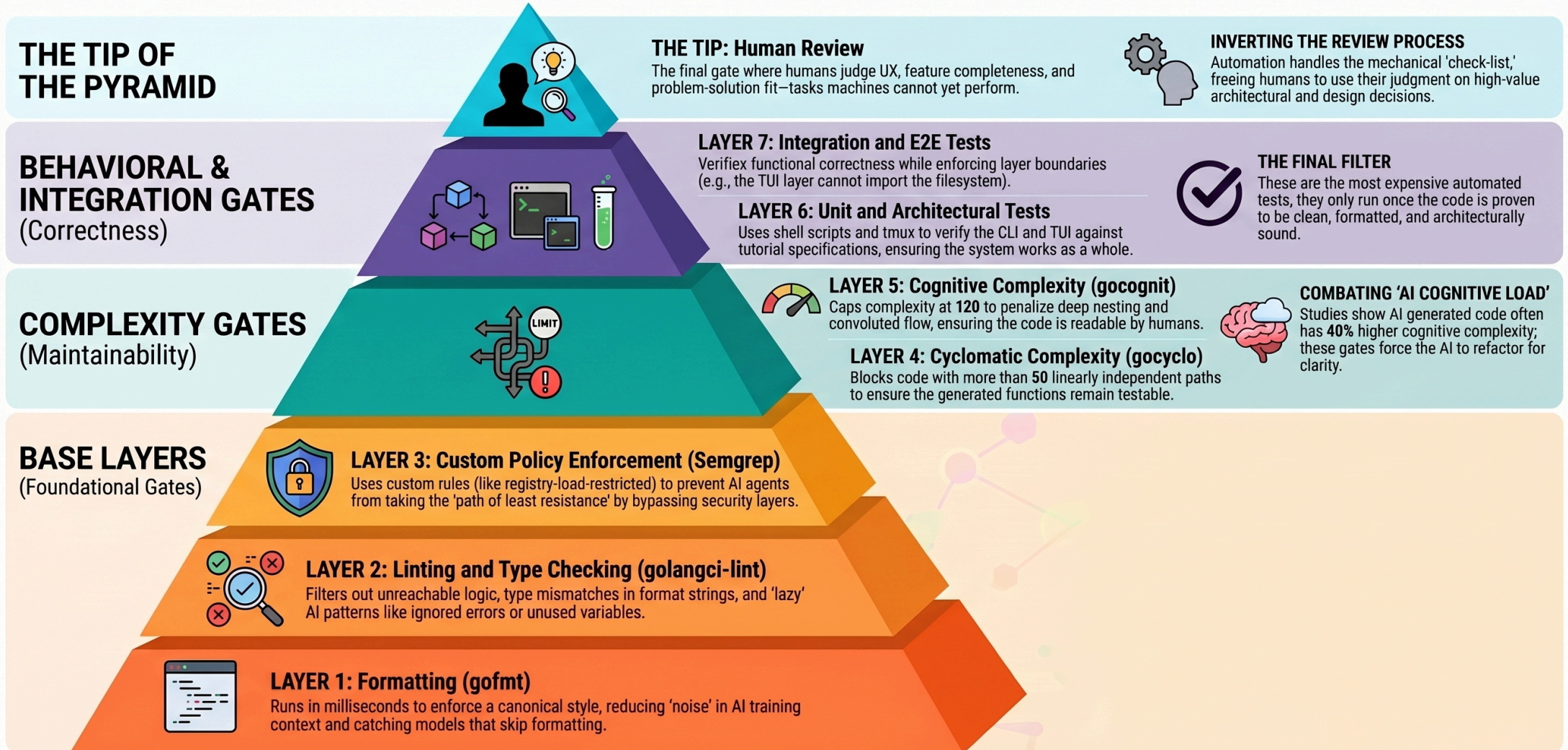
The Verification Gap

- 42% of code is AI-generated
- 96% of developers don't fully trust it
- 48% validate all of it

The AI Validation Pyramid

The AI Validation Pyramid: Safeguarding Machine-Generated Code

To explain how the 'Guard' CI pipeline uses a tiered, automated structure to filter 'AI slop' and **protect human reviewers** from **low-quality** machine-generated code.



Internal vs External Quality

- Tests validate intended behavior
- Guardrails enforce internal code quality

You Need Both

- Code itself is context - which is why quality matters.
- Tests and guardrails provide feedback that becomes part of the context.

Why It Works

The agent self-corrects when the environment talks back.

The Shift

- I stopped reviewing code
- I review behavior and AI-generated reports
- Every issue becomes a new test or guardrail

Bonus Patterns - Skills I Use

- **Beyond SOLID Principles** - structural sanity
- **Archibald** - architecture review
- **Security Review** - adversarial assumptions
- **Adversarial Review** - try to crash your own software

Augmenting Reviews with AI-Assisted Reviews

- **Pattern skills** - SOLID, common smells, etc.
- **Specialized agents** - security, input validation, performance, etc.
- **Compliance agents** - check implementation against spec / intent / best practices.

One agent finds issues. A second agent says which ones are real.

How to Get Started

1. **Bootstrap:** Init the guardrails with defaults; verify they pass on a green baseline.
2. **Define and document behavior:** Expected behavior as tutorials with examples, inputs, outputs and steps.
3. **Decide and document the architecture:** Components, dependencies and constraints.
4. **Guardrails:** Convert behaviors and architecture into tests and guardrails.
5. **Implementation:** In an environment where the guardrails provide feedback; lock the tests.
6. **Review:** Analyze the results and issues with agents/skills until satisfied. → Update guardrails/add tests.
7. **Iterate:** GOTO 2, add new features by repeating the loop.

Thank You!

Github: [github.com/florianbuetow/...](https://github.com/florianbuetow/)

- guard
- ai-guardrails
- claude-code

LinkedIn: [linkedin.com/in/florianbuetow](https://www.linkedin.com/in/florianbuetow)

